

Derivative Free One Dimensional Minimization

Methods of Optimization and Decision Making

Individual programming project | September 2026

Student _____ Group _____

Purpose and main result

I compare five derivative-free methods using the number of objective function evaluations needed to meet a common accuracy target. The program separates the objective function from every search method, so the same code can solve a new problem during the practical class.

For an absolute location tolerance of 0.001, golden section and Fibonacci each use 19 total evaluations in the selected example, compared with 7002 for uniform search. All five methods satisfy the requested error bound. The experiments also compare one long Fibonacci search with several shorter searches.

Problem formulation

$$\min_{x \in [a, b]} f(x), [a, b] = [-2, 5]$$
$$f(x) = 1 + (x - \sqrt{2})^2 + 0.2(x - \sqrt{2})^4$$

The example is a nonquadratic polynomial after a shift of the variable. Its known answer permits independent validation; the search algorithms never receive that answer.

$$x^* = \sqrt{2}, f(x^*) = 1$$

Accuracy convention

$$\delta \in \{0.1, 0.01, 0.001\}$$

Here delta denotes an absolute error in the minimizer location, not an error in the objective value. Every method returns the midpoint of a retained interval that contains the minimum.

$$b_f - a_f \leq 2\delta \Rightarrow \left| \frac{a_f + b_f}{2} - x^* \right| \leq \delta$$

The guarantee assumes strict unimodality and reliable function comparisons. Reported decimal values are rounded for readability; the bound applies to the unrounded midpoint.

Demonstration of unimodality

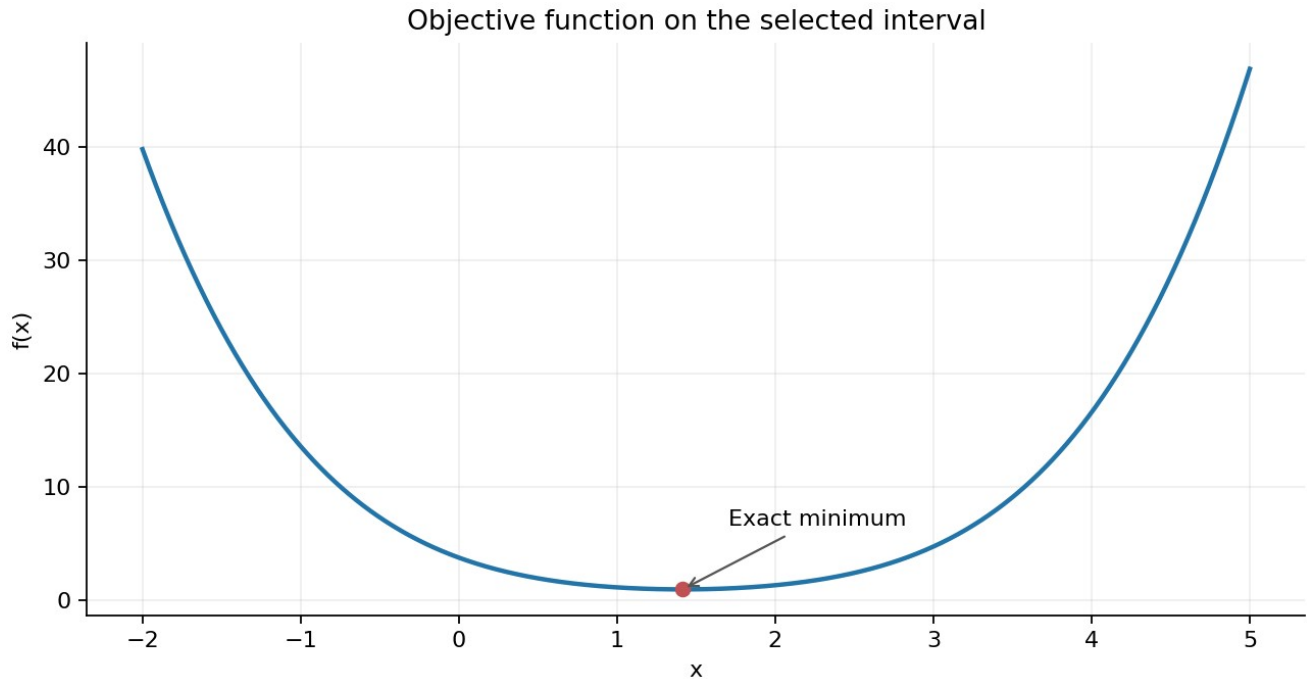


Figure 1 The function decreases toward the unique minimum and increases after it.

The graph shows a single valley on the selected interval. The function decreases before the minimum and increases after the minimum. Therefore, it is unimodal on the selected interval. The following argument supports this observation without using derivatives.

An argument based on distance

Let d be the nonnegative distance from x to the square root of two. The function value is $g(d) = 1 + d^2 + 0.2 d^4$. For any two distances with d_2 greater than d_1 , the difference is positive:

$$g(d_2) - g(d_1) = (d_2^2 - d_1^2) \left(1 + 0.2(d_2^2 + d_1^2) \right) > 0$$

The first factor is positive and the second factor is greater than zero. Thus, the value increases strictly with distance from the reference point. Moving from the left toward that point reduces the distance; moving to the right after it increases the distance. This establishes strict unimodality without differentiation.

Both added terms are nonnegative and become zero together only at the reference point, so the minimum value is exactly one. A sampled graph alone would not establish unimodality for an arbitrary new objective.

Description of algorithms

Uniform search

$$n = \text{ceil}\left(\frac{b-a}{\delta}\right), h = \frac{b-a}{n} \leq \delta$$

Evaluate the $n + 1$ grid points, including both endpoints. Find the smallest sampled value and retain the interval between its neighbors, clipped at an endpoint when necessary. Its width is at most two grid steps. Return the midpoint. This is an exhaustive grid baseline.

Sequential scan for a local extremum

Use the same grid and move from left to right. At the first non-decreasing value, retain the interval from the point two positions earlier to the current point. If the function decreases across the entire grid, retain the last grid segment. This project uses the sequential grid-scan definition of local-extremum search.

Dichotomy

$$x_1 = \frac{a+b}{2} - \frac{s}{2}, x_2 = \frac{a+b}{2} + \frac{s}{2}, s = 0.1 \delta$$

If the first function value is no greater than the second, keep the interval ending at x_2 ; otherwise keep the interval starting at x_1 . Each reduction uses two new evaluations. The separation s is the full distance between the probes and is smaller than the target width.

Golden section

$$\rho = \frac{\sqrt{5}-1}{2}, x_1 = b - \rho(b-a), x_2 = a + \rho(b-a)$$

Compare the two values and discard the side that cannot contain the minimizer. The golden ratio places one retained probe at the correct position for the next reduction, so its value can be reused. After initialization, each additional reduction needs one new value [1]. No unused probe is evaluated after the stopping condition is met.

Fibonacci search

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

Choose the horizon before the search. At remaining index m , use the ratios $F(m-2)/F(m)$ and $F(m-1)/F(m)$. Reuse the retained probe and reduce m after each comparison. The final coincident ideal probes are separated by a small positive perturbation. The implementation includes that perturbation when selecting the horizon [2].

Program structure and evaluation accounting

Choice of programming language

I selected Python because it provides concise, readable implementations of numerical algorithms, convenient tools for visualization and numerical experiments, and a clear separation between the objective function and search methods. The algorithms use only the standard library. Matplotlib is used for figures. A ready-made optimization solver is not used.

File	Role
problem.py	Objective, interval, accuracies, reference metadata
algorithms.py	Five methods, counter, result and bracket history
main.py	Experiments, formatted tables and JSON output
plots.py	Objective, evaluation, convergence and validation figures
research.py	Comparison at a fixed budget of search evaluations
test_algorithms.py	Accuracy, interval, counter and input validation
Optimization_Demo.ipynb	Optional notebook using the same source modules

A consistent counter

Each method receives a fresh EvaluationCounter. Its count increases only when the objective is actually called. Assigning or reusing a stored function value does not increase the count. One additional evaluation is made at the final midpoint to report its function value, even if that midpoint was previously sampled.

$$N_{total} = N_{search} + 1$$

Plotting and independent validation calls are excluded from both columns. Every method and every requested accuracy starts a separate run. No values are shared between competing algorithms. Restarted Fibonacci retains values within each stage but discards them at a restart; only the final stage is followed by a reporting call.

Stopping and numerical details

Uniform and sequential search use a grid step no greater than delta. Dichotomy uses a separation of 0.1 delta. Golden section stops before making an unnecessary new evaluation. Fibonacci selects the smallest index n of at least three that meets the following bound:

$$L_f \leq \frac{L_0}{F_n} + \varepsilon = \frac{1.01 L_0}{F_n} \leq 2 \delta, \varepsilon = \frac{0.01 L_0}{F_n}$$

A Fibonacci pass uses n - 1 search evaluations. The distinct final probe avoids the incorrect comparison of a point with itself. Invalid intervals, nonpositive tolerances, nonfinite values and impractically large grids produce explicit errors.

Computational experiments and results

The following results were generated by running the supplied Python program on the selected interval. The environment used Python 3.12.14. The objective is deterministic, so the comparison does not depend on random sampling or runtime measurements.

delta	Method	Estimated x	f at estimate	Search calls	Total calls
0.1	Uniform search	1.40	1.000	71	72
0.1	Sequential scan	1.40	1.000	36	37
0.1	Dichotomy	1.45	1.001	12	13
0.1	Golden section	1.38	1.001	9	10
0.1	Fibonacci	1.37	1.002	9	10
0.01	Uniform search	1.410	1.0000	701	702
0.01	Sequential scan	1.410	1.0000	343	344
0.01	Dichotomy	1.411	1.0000	18	19
0.01	Golden section	1.412	1.0000	14	15
0.01	Fibonacci	1.407	1.0000	13	14
0.001	Uniform search	1.4140	1.00000	7001	7002
0.001	Sequential scan	1.4140	1.00000	3416	3417
0.001	Dichotomy	1.4137	1.00000	24	25
0.001	Golden section	1.4146	1.00000	18	19
0.001	Fibonacci	1.4146	1.00000	18	19

Table 1 Results for a shared absolute error target in x. Total calls include exactly one final reporting evaluation.

Checking the numerical answers

Every returned interval contains the exact minimizer, and every midpoint is within its requested tolerance. The largest observed location errors across the five methods are 0.04085, 0.006959 and 0.0005161 for tolerances 0.1, 0.01 and 0.001 respectively.

Displayed objective values become equal after rounding near the minimum. This is expected: the objective changes slowly there. The location error and interval width provide a more informative check than comparing printed function values alone. Full precision is retained in results.json.

Effect of requesting higher accuracy

Reducing delta increases the number of evaluations for every method in these experiments. The answers move closer to the reference as the tolerance decreases here. In general, a smaller guaranteed interval does not require the actual error to decrease monotonically on every individual run.

Comparison of methods

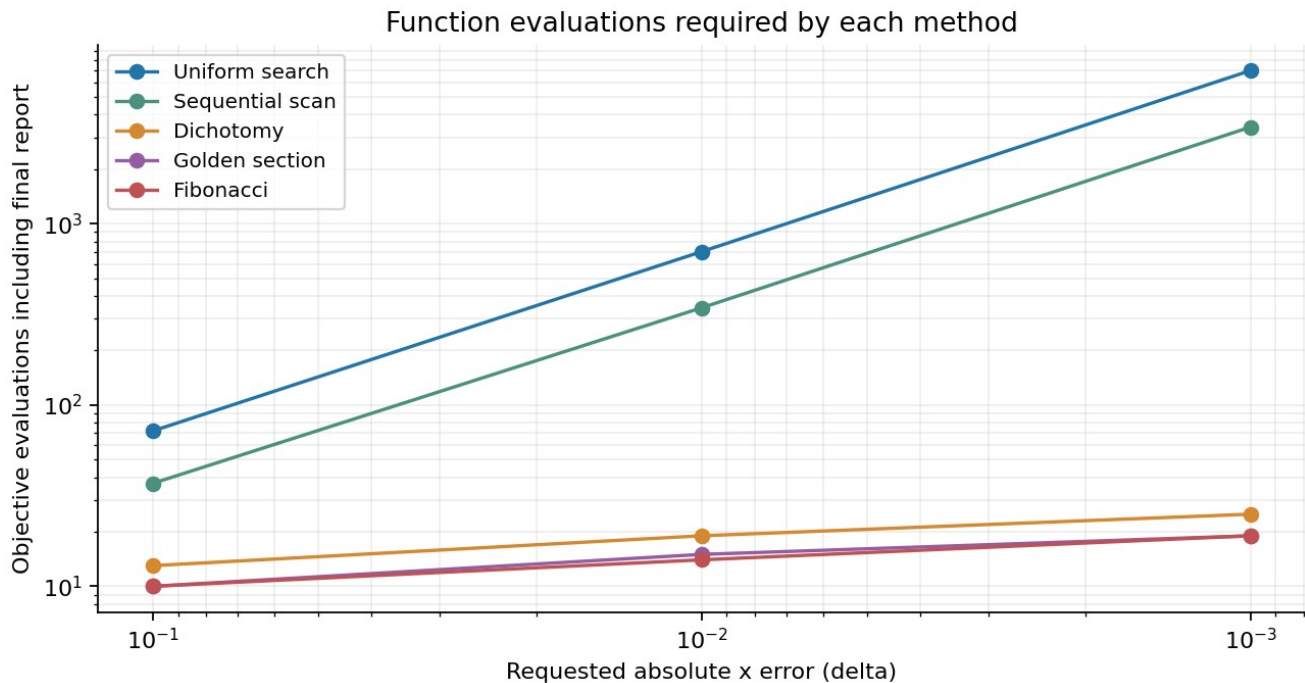


Figure 2 Both axes use logarithmic scales. Accuracy increases from left to right.

Uniform search uses 71, 701 and 7001 search evaluations. A tenfold reduction of the tolerance requires approximately ten times as many grid samples. The sequential scan stops earlier because it reaches the valley before the right endpoint, but its worst-case cost is still linear in the number of grid points.

Dichotomy uses 12, 18 and 24 search evaluations. It nearly halves the interval at each reduction, but pays for two new function values. Golden section uses 9, 14 and 18 search evaluations, while Fibonacci uses 9, 13 and 18. Their reuse of one probe gives a lower cost at the same requested tolerance.

At delta 0.001, golden section and Fibonacci require about 369 times fewer total evaluations than uniform search in this example. Fibonacci saves one evaluation relative to golden section at delta 0.01; the counts tie at the other two tolerances.

Advantages and limitations

Uniform search is easy to inspect but expensive at fine accuracy. Sequential scanning is also simple and can stop early, although its cost depends on the minimum's location and scan direction. Dichotomy is straightforward but requires a suitable probe separation. Golden section has a constant ratio and can continue without a predetermined final horizon. Fibonacci plans its horizon to use a finite evaluation budget efficiently, at the cost of extra indexing and care in the last step.

Efficiency here means evaluations needed to meet the same error bound. A method can satisfy that bound with fewer calls while another happens to produce a closer estimate on one run. These are different comparisons.

Additional research on Fibonacci search

Golden section and a finite Fibonacci horizon

Let L_0 be the initial width and M the number of search evaluations, excluding the final reporting call. With M at least two, golden section performs $M - 1$ reductions. A Fibonacci pass has index $n = M + 1$ in this implementation. Therefore:

$$L_G = L_0 \rho^{M-1}, L_F \leq \frac{1.01 L_0}{F_{M+1}}$$

Classical Fibonacci search is designed for a fixed evaluation budget and has a minimax interval advantage under the usual unimodal comparison assumptions [3]. The implemented positive terminal perturbation adds a one percent margin to the ideal Fibonacci bound. The constant-ratio golden method is simpler when the eventual budget is not fixed.

$$F_n \approx \frac{\phi^n}{\sqrt{5}}, \phi = \frac{1 + \sqrt{5}}{2} = \rho^{-1}$$

This approximation explains why the two methods have similar geometric convergence [2]. Substituting it into the bounds above gives a Fibonacci-to-golden width ratio approaching 1.01 times the square root of five divided by phi squared, about 0.863. This is a comparison of bounds at the same budget; it does not promise a smaller actual location error on every function.

One long search and repeated short searches

For r equal stages with Fibonacci index m , each stage costs $m - 1$ search calls. Restarting on the retained interval and discarding stored values gives:

$$M = r(m - 1), L_R \leq L_0 \left(\frac{1.01}{F_m} \right)^r$$

For two F8 stages, the total search budget is 14. One long F15 pass has the same budget. With initial width seven, the bound is about 0.011590 for the long pass and 0.016192 for the two short passes. Planning the whole horizon therefore gives a tighter bound in this matched comparison.

Strategy	Search calls	Observed width	Width bound
Golden section	14	0.013436	0.013436
One Fibonacci F15	14	0.011590	0.011590
Two Fibonacci F8 stages	14	0.015873	0.016192

Table 2 The research.py experiment uses exactly 14 search calls in each row. An extra midpoint reporting call is excluded from this comparison.

Restart experiments and interval validation

A second experiment fixes the requested accuracy. Each restarted strategy uses a maximum stage index of 6 or 8; its last stage may be shorter. It continues on the retained interval, rather than returning to the original interval.

delta	Strategy	Stages	Search calls	Total calls
0.1	Fibonacci	1	9	10
0.1	Fibonacci restart F6	2	9	10
0.1	Fibonacci restart F8	2	9	10
0.01	Fibonacci	1	13	14
0.01	Fibonacci restart F6	3	15	16
0.01	Fibonacci restart F8	2	14	15
0.001	Fibonacci	1	18	19
0.001	Fibonacci restart F6	4	20	21
0.001	Fibonacci restart F8	3	20	21

Table 3 One long pass ties the restarted strategies at delta 0.1 and uses fewer calls at the two finer accuracies. Integer horizons and the final comparison can produce ties. Restarting can be useful when the objective or the budget changes, but it has no demonstrated evaluation benefit for this fixed problem.

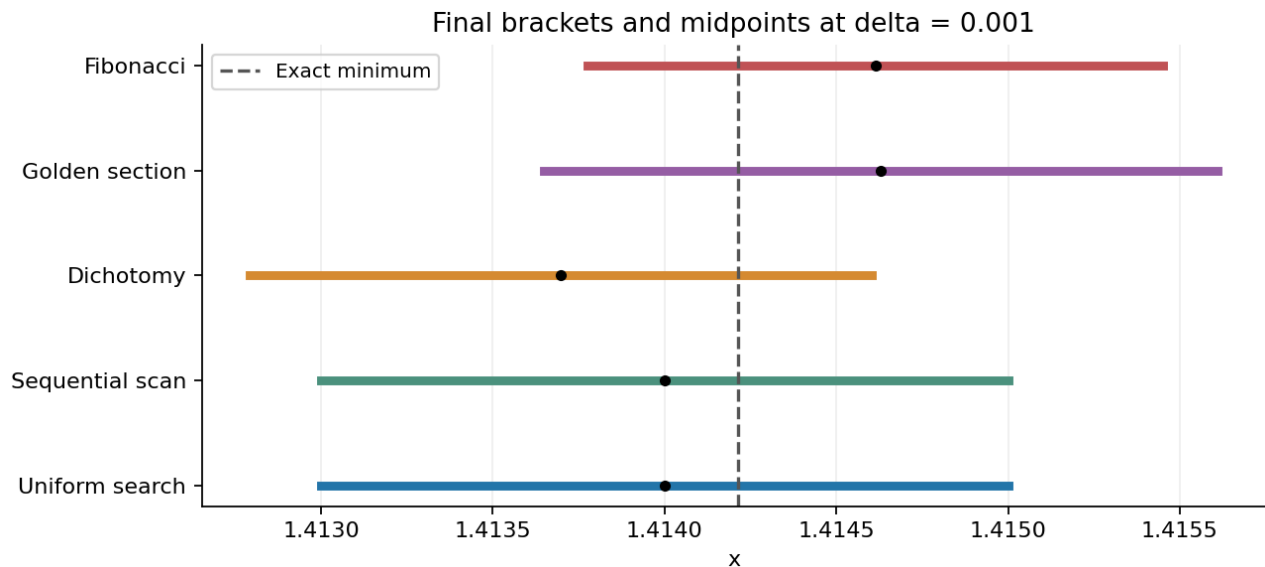


Figure 3 At delta 0.001, all retained intervals contain the exact minimizer. Dots mark their midpoints.

The graph and the exact error checks agree. The methods need not return identical points; each only has to keep the minimizer inside an interval narrow enough to satisfy the common tolerance.

Validation and use during the practical class

Independent program checks

Five automated test groups pass. They include 269 complete accuracy and accounting scenarios and 65 checks of Fibonacci horizons and final probes. The scenarios use shifted quadratics, an absolute-value function, a reciprocal example, a quartic, asymmetric objectives, symmetric ties and minima at either endpoint.

An independent wrapper counts actual objective calls. The tests compare that count with the reported totals and check every retained interval for containment and nesting. Separate checks cover invalid inputs and an initial interval that already meets the requested accuracy. The notebook's six code cells were also executed in sequence.

Demonstrating a new problem

Edit `objective_function`, `A`, `B`, `DELTAS` and `DESCRIPTION` in `problem.py`. Set both reference values to `None` when the new exact answer is unknown. Save the file, then run the following command from the project folder:

```
python main.py --output results_live
```

The new tables and figures are written to a separate folder. No algorithm code needs to change. Verify that the new function is finite and unimodal on the supplied interval. The report itself documents the original experiment and is not automatically rewritten by that command.

Conclusion

All five methods solve the example to the requested accuracy. Golden section and Fibonacci are the most economical in function evaluations in these tests. A single planned Fibonacci horizon is preferable to the tested restart strategies when the objective and final accuracy are known in advance. The shared interface, explicit error convention and verified counters make the project reusable during the practical class.

Scope

The search guarantees apply to deterministic strictly unimodal functions with reliable comparisons. A noisy or multimodal objective can invalidate interval elimination. At very small tolerances, floating-point rounding can make distinct values indistinguishable. The tested tolerances avoid that issue for the selected example.

References

[1] University of Illinois Urbana-Champaign. Optimization. CS 357, Spring 2023.

https://courses.physics.illinois.edu/cs357/sp2023/notes/ref-15-opt_nd.html

[2] E. Bertolazzi. One-Dimensional Minimization. University of Trento, May 2008.

https://e.bertolazzi.dii.unitn.it/corso-PHD/AA2007_2008_UOPT/lucidi/slides-m1D-1x2.pdf

[3] M. Avriel and D. J. Wilde. Optimality Proof for the Symmetric Fibonacci Search Technique. The Fibonacci Quarterly, 4(3), 1966, 265-269. <https://www.fq.math.ca/Scanned/4-3/avriel.pdf>

Program code

problem.py | lines 1 to 19

The following listings reproduce the objective and complete optimizer module. The driver, plotting, research and test modules are supplied as runnable source files in the project archive.

```
"""Edit this file when the lecturer gives you a new problem."""
import math

def objective_function(x):
    """A nonquadratic, strictly unimodal example."""
    t = x - math.sqrt(2.0)
    return 1.0 + t**2 + 0.2 * t**4

A = -2.0
B = 5.0
DELTAS = (0.1, 0.01, 0.001)
DESCRIPTION = "f(x) = 1 + (x - sqrt(2))^2 + 0.2*(x - sqrt(2))^4"

# Validation metadata ONLY. Algorithms never receive these values.
# Set both to None when the new problem's exact answer is unknown.
REFERENCE_X = math.sqrt(2.0)
REFERENCE_F = 1.0
```

Optimization algorithm code

algorithms.py | lines 1 to 53

```
"""Derivative-free interval minimization using only the standard library.

Assumptions: finite deterministic values, strict decrease before the unique
minimum and strict increase after it. A boundary minimum is also allowed.
delta means absolute error in x. All methods return a bracket of width
at most 2*delta, and report its midpoint. Plotting is not counted.
"""

from dataclasses import dataclass, asdict
import math

class EvaluationCounter:
    """Count actual objective calls, including one final reporting call."""

    def __init__(self, function, limit=2_000_000):
        self.function = function
        self.calls = 0
        self.limit = limit

    def __call__(self, x):
        if self.calls >= self.limit:
            raise ValueError("Evaluation limit exceeded; increase delta.")
        self.calls += 1
        value = float(self.function(x))
        if not math.isfinite(value):
            raise ValueError(f"Objective is not finite at x={x!r}.")
        return value

@dataclass
class Result:
    method: str
    delta: float
    x: float
    fx: float
    left: float
    right: float
    search_evaluations: int
    total_evaluations: int
    stages: int
    history: list

    @property
    def error_bound(self):
        return (self.right - self.left) / 2.0

    def to_dict(self):
        return {**asdict(self), "error_bound": self.error_bound}

def _setup(function, a, b, delta):
    if not all(math.isfinite(v) for v in (a, b, delta)):
        raise ValueError("a, b and delta must be finite.")
```

Optimization algorithm code

algorithms.py | lines 54 to 106

```
    if a >= b or delta <= 0 or not math.isfinite(b - a):
        raise ValueError("Require a < b, delta > 0 and a finite interval.")
    if delta < 32 * max(math.ulp(a), math.ulp(b)):
        raise ValueError("delta is too small for this floating-point scale.")
    counter = EvaluationCounter(function)
    history = [(0, a, b)] # (search calls, left endpoint, right endpoint)
    return counter, history

def _finish(name, counter, a, b, delta, history, stages=1):
    if b - a > 2 * delta * (1 + 1e-12):
        raise ArithmeticError("Requested bracket width was not reached.")
    x = a + (b - a) / 2
    search_calls = counter.calls
    fx = counter(x) # Always one reporting call, even if x was sampled.
    return Result(name, delta, x, fx, a, b, search_calls,
                  counter.calls, stages, history)

def _grid(a, b, delta, limit):
    n = max(1, math.ceil((b - a) / delta))
    if n + 2 > limit:
        raise ValueError("Grid too large; increase delta or omit grid methods.")
    return n, (b - a) / n

def uniform_search(function, a, b, delta):
    """Sample the whole grid; bracket its best point by its neighbors."""
    f, history = _setup(function, a, b, delta)
    if b - a <= 2 * delta:
        return _finish("Uniform search", f, a, b, delta, history)
    n, h = _grid(a, b, delta, f.limit)
    best_i, best_value = 0, f(a)
    for i in range(1, n + 1):
        value = f(b if i == n else a + i * h)
        if value < best_value:
            best_i, best_value = i, value
    left = a + max(0, best_i - 1) * h
    right = min(b, a + min(n, best_i + 1) * h)
    history.append((f.calls, left, right))
    return _finish("Uniform search", f, left, right, delta, history)

def sequential_search(function, a, b, delta):
    """Scan left to right; stop at the first non-decreasing grid value."""
    f, history = _setup(function, a, b, delta)
    if b - a <= 2 * delta:
        return _finish("Sequential scan", f, a, b, delta, history)
    n, h = _grid(a, b, delta, f.limit)
    previous = f(a)
    for i in range(1, n + 1):
        x = b if i == n else a + i * h
        current = f(x)
```

Optimization algorithm code

algorithms.py | lines 107 to 159

```
        if current >= previous:
            left, right = a + max(0, i - 2) * h, x
            break
        previous = current
    else:
        left, right = a + (n - 1) * h, b
    history.append((f.calls, left, right))
    return _finish("Sequential scan", f, left, right, delta, history)

def dichotomy_search(function, a, b, delta):
    """Compare two points on either side of the interval midpoint."""
    f, history = _setup(function, a, b, delta)
    separation = 0.1 * delta # FULL distance between the two probes.
    while b - a > 2 * delta:
        midpoint = a + (b - a) / 2
        x1, x2 = midpoint - separation / 2, midpoint + separation / 2
        if not a < x1 < x2 < b:
            raise ArithmeticError("Dichotomy probes cannot be represented.")
        y1, y2 = f(x1), f(x2)
        if y1 <= y2:
            b = x2
        else:
            a = x1
        history.append((f.calls, a, b))
    return _finish("Dichotomy", f, a, b, delta, history)

def golden_section_search(function, a, b, delta):
    """Reuse one old function value after each interval reduction."""
    f, history = _setup(function, a, b, delta)
    if b - a <= 2 * delta:
        return _finish("Golden section", f, a, b, delta, history)
    ratio = (math.sqrt(5) - 1) / 2
    x1, x2 = b - ratio * (b - a), a + ratio * (b - a)
    y1, y2 = f(x1), f(x2)
    while b - a > 2 * delta:
        if y1 <= y2:
            b, x2, y2 = x2, x1, y1
            history.append((f.calls, a, b))
            if b - a <= 2 * delta:
                break # Do not evaluate a point that will not be used.
            x1 = b - ratio * (b - a)
            if not a < x1 < x2 < b:
                raise ArithmeticError("Golden-section probes collapsed.")
            y1 = f(x1)
        else:
            a, x1, y1 = x1, x2, y2
            history.append((f.calls, a, b))
            if b - a <= 2 * delta:
                break
            x2 = a + ratio * (b - a)
            if not a < x1 < x2 < b:
```

Optimization algorithm code

algorithms.py | lines 160 to 212

```
        raise ArithmeticError("Golden-section probes collapsed.")
        y2 = f(x2)
    return _finish("Golden section", f, a, b, delta, history)

def _fibonacci_numbers(n):
    numbers = [0, 1]
    while len(numbers) <= n:
        numbers.append(numbers[-1] + numbers[-2])
    return numbers

def _fibonacci_index(length, target):
    # The final distinct probe adds at most 1% to the ideal final width.
    numbers = [0, 1, 1, 2]
    while 1.01 * length / numbers[-1] > target:
        numbers.append(numbers[-1] + numbers[-2])
    return len(numbers) - 1

def _fibonacci_pass(f, a, b, n, history):
    """Use n-1 search calls; final width <= 1.01*(initial width)/F[n]."""
    numbers = _fibonacci_numbers(n)
    unit = (b - a) / numbers[n]
    epsilon = 0.01 * unit
    if n == 3:
        retained = a + (b - a) / 2
        retained_value = f(retained)
    else:
        x1 = a + numbers[n - 2] * unit
        x2 = a + numbers[n - 1] * unit
        y1, y2 = f(x1), f(x2)
        m = n
        while m > 3:
            if y1 <= y2:
                b, x2, y2 = x2, x1, y1
                m -= 1
                history.append((f.calls, a, b))
                if m == 3:
                    retained, retained_value = x2, y2
                    break
                x1 = a + numbers[m - 2] / numbers[m] * (b - a)
                y1 = f(x1)
            else:
                a, x1, y1 = x1, x2, y2
                m -= 1
                history.append((f.calls, a, b))
                if m == 3:
                    retained, retained_value = x1, y1
                    break
                x2 = a + numbers[m - 1] / numbers[m] * (b - a)
                y2 = f(x2)
    # At the terminal Fibonacci ratio the two ideal probes coincide.
```

Optimization algorithm code

algorithms.py | lines 213 to 253

```
# A distinct nearby probe is necessary; comparing x with itself is wrong.
probe = retained + epsilon
if not a < retained < probe < b:
    raise ArithmeticError("Fibonacci final probe cannot be represented.")
if retained_value <= f(probe):
    b = probe
else:
    a = retained
history.append((f.calls, a, b))
return a, b

def fibonacci_search(function, a, b, delta):
    """Choose one Fibonacci horizon from the requested tolerance."""
    f, history = _setup(function, a, b, delta)
    if b - a > 2 * delta:
        n = _fibonacci_index(b - a, 2 * delta)
        a, b = _fibonacci_pass(f, a, b, n, history)
    return _finish("Fibonacci", f, a, b, delta, history)

def restarted_fibonacci(function, a, b, delta, stage_index=8):
    """Research variant: restart shorter horizons on each retained bracket.

    Reuse values inside a stage, but deliberately discard them at a restart.
    Do not evaluate intermediate midpoints. Report only the final midpoint.
    """
    if not isinstance(stage_index, int) or stage_index < 3:
        raise ValueError("stage_index must be an integer >= 3.")
    f, history = _setup(function, a, b, delta)
    stages = 0
    while b - a > 2 * delta:
        n = min(stage_index, _fibonacci_index(b - a, 2 * delta))
        a, b = _fibonacci_pass(f, a, b, n, history)
        stages += 1
    name = f"Fibonacci restart F{stage_index}"
    return _finish(name, f, a, b, delta, history, stages)

METHODS = (uniform_search, sequential_search, dichotomy_search,
            golden_section_search, fibonacci_search)
```